



REAL TIME SYSTEM AND INTERNET OF THINGS FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

SmartGuard: Industrial Predictive Maintenance System

GROUP 22

Daffa Hardhan	2306161763
Siti Amalia Nurfaidah	2306161851
Raka Arrayan Muttaqien	2306161800
Naufal Hadi Rasikhin	2306231366

PREFACE

Proyek *SmartGuard: Industrial Predictive Maintenance System* dibuat untuk menjawab masalah umum yang sering terjadi di dunia industri, yaitu kerusakan mendadak pada mesin. Ketika mesin rusak tanpa tanda-tanda sebelumnya, proses produksi bisa terhenti dan menimbulkan kerugian besar. Melihat kebutuhan akan sistem yang dapat mendeteksi gejala kerusakan lebih awal, kami berusaha merancang sebuah alat berbasis IoT yang mampu memantau kondisi mesin secara otomatis dan memberikan respons cepat ketika terjadi keadaan berbahaya.

Dalam proyek ini, kami menggabungkan beberapa teknologi seperti sensor getaran, *Bluetooth Low Energy* (BLE), FreeRTOS, dan *cloud server* untuk membangun sistem *predictive maintenance* yang sederhana. Sensor akan mengirimkan data secara nirkabel ke *Gateway*, lalu *Gateway* memproses data tersebut dan mengambil keputusan secara langsung tanpa harus bergantung pada internet. Dengan cara ini, mesin tetap terlindungi meskipun jaringan WiFi bermasalah.

Laporan ini disusun untuk menjelaskan bagaimana sistem kami dirancang, bagaimana proses pembuatannya, serta bagaimana hasil pengujian yang telah dilakukan. Kami berharap penjelasan dalam laporan ini mudah dipahami dan dapat menjadi referensi bagi siapa pun yang ingin mempelajari atau mengembangkan sistem IoT serupa. Terima kasih kepada semua pihak yang telah membantu kami selama proses pengerjaan proyek ini.

Depok, Desember 7, 2025

Grup 22

TABLE OF CONTENTS

CHAPTER 1.....	4
INTRODUCTION.....	4
1.1 PROBLEM STATEMENT.....	4
1.3 ACCEPTANCE CRITERIA.....	5
1.4 ROLES AND RESPONSIBILITIES.....	5
1.5 TIMELINE AND MILESTONES.....	5
CHAPTER 2.....	7
IMPLEMENTATION.....	7
2.1 HARDWARE DESIGN AND SCHEMATIC.....	7
2.2 SOFTWARE DEVELOPMENT.....	7
2.3 HARDWARE AND SOFTWARE INTEGRATION.....	8
CHAPTER 3.....	9
TESTING AND EVALUATION.....	9
3.1 TESTING.....	9
3.2 RESULT.....	9
3.3 EVALUATION.....	10
CHAPTER 4.....	11
CONCLUSION.....	11

CHAPTER 1

INTRODUCTION

1.1 PROBLEM STATEMENT

Dalam industri manufaktur, gangguan operasional akibat kegagalan mesin (*unexpected downtime*) dapat menyebabkan kerugian finansial, penurunan produktivitas, serta potensi kerusakan mesin secara permanen. Sistem pemantauan kondisi mesin yang digunakan saat ini memiliki beberapa kelemahan, antara lain:

1. Ketergantungan pada koneksi WiFi terpusat, sehingga saat jaringan terputus, sistem tidak mampu memberikan peringatan atau mengambil tindakan keselamatan secara mandiri.
2. Pemasangan sensor berbasis kabel yang sulit, terutama pada mesin bergerak atau bergetar tinggi.
3. Kurangnya respons *real-time* dari sistem berbasis *cloud* yang menyebabkan tindakan keselamatan terlambat.
4. Ketidakmampuan sistem konvensional untuk melakukan tindakan otomatis, seperti memutus daya mesin ketika getaran mencapai level berbahaya.
5. Minimnya efisiensi daya pada sensor, yang menyebabkan frekuensi pengisian baterai tinggi dan mengganggu kontinuitas pemantauan.

Akibatnya, sistem *monitoring* tidak dapat digunakan sebagai solusi *predictive maintenance* yang andal, khususnya pada lingkungan industri yang membutuhkan respons cepat dan independensi dari internet. Tanpa adanya sistem otonom yang andal, strategi perawatan mesin cenderung tetap bersifat reaktif daripada prediktif, yang pada akhirnya memperbesar biaya perbaikan. Kondisi ini juga meningkatkan risiko keselamatan kerja bagi operator yang berada di area sekitar mesin yang berpotensi mengalami malfungsi fatal tanpa adanya peringatan dini.

1.2 PROPOSED SOLUTION

SmartGuard: Industrial Predictive Maintenance System mengusulkan penerapan sistem *Predictive Maintenance* berbasis IoT dengan arsitektur *Hybrid Network* yang memisahkan fungsi antara *Sensor Node* dan *Gateway Node*. *Sensor Node* dirancang sebagai perangkat cerdas berdaya rendah yang memanfaatkan *Direct Register Access* pada sensor MPU6050 untuk mengakuisisi data getaran dan suhu mesin secara presisi tanpa membebani memori. Data parameter operasional tersebut dikirimkan ke *Gateway* melalui notifikasi *Bluetooth Low Energy* (BLE) yang hemat energi. Untuk memaksimalkan masa pakai baterai, *Sensor Node* menerapkan algoritma manajemen daya berbasis *Deep Sleep* yang aktif otomatis jika mesin tidak menunjukkan aktivitas selama 30 detik, dengan siklus bangun periodik setiap 10 detik.

Di sisi lain, *Gateway Node* menggunakan ESP32 dengan sistem operasi FreeRTOS untuk memastikan pemrosesan data secara *real-time* dan bebas hambatan melalui pembagian tugas multitasking. *Gateway* menganalisis data getaran yang diterima, memicu *Relay* untuk memutus daya mesin dalam situasi berbahaya, serta mengaktifkan alarm secara lokal tanpa ketergantungan pada koneksi internet. Selain fungsi keselamatan lokal, *Gateway* mengirimkan log ke *Cloud* melalui API Flask, sekaligus menampilkan notifikasi status mesin di aplikasi Blynk sebagai alat pemantauan jarak jauh. Integrasi komponen tersebut menghasilkan solusi industri yang responsif, dan berfungsi meskipun jaringan WiFi mengalami gangguan.

1.3 ACCEPTANCE CRITERIA

Adapun kriteria keberhasilan dari proyek ini adalah sebagai berikut:

1. **Respons Cepat (*Real-Time Response*):** *Gateway* harus dapat memutus daya mesin melalui *Relay* dalam waktu kurang dari 1 detik setelah nilai getaran melebihi ambang batas 5.5 m/s^2 .
2. **Koneksi Nirkabel Stabil (*Wireless Reliability*):** Pengiriman data dari *Sensor Node* ke *Gateway* melalui *Bluetooth Low Energy* (BLE) harus tetap stabil pada jarak minimal 5 meter.
3. **Efisiensi Daya Sensor (*Power Efficiency*):** *Sensor Node* harus otomatis masuk ke mode *Deep Sleep* setelah tidak mendeteksi aktivitas mesin selama 30 detik, untuk menghemat baterai.

4. **Fungsi Cloud Berjalan Normal (*Cloud Integration*):** Data getaran dan suhu harus tercatat di database Flask, serta notifikasi bahaya (*DANGER*) muncul di aplikasi Blynk ketika WiFi tersedia.
5. **Gateway Tetap Stabil Tanpa WiFi (*Multitasking Stability*):** *Gateway* harus tetap bekerja normal dalam memproses data Sensor dan mengendalikan *Relay*, meskipun koneksi WiFi sedang terputus.

1.4 ROLES AND RESPONSIBILITIES

Berikut adalah pembagian peran setiap anggota kelompok dalam proyek ini:

Roles	Responsibilities	Person
Role 1	Bertanggung jawab atas perakitan fisik (<i>wiring</i>), integrasi antara perangkat keras dan lunak, serta pelaksanaan pengujian sistem menyeluruh (<i>running test</i>) untuk memastikan reliabilitas alat.	Daffa Hardhan
Role 2	Bertanggung jawab atas pengembangan kode <i>Sensor Node</i> , termasuk implementasi akses register I2C manual pada MPU6050, manajemen daya (<i>Deep Sleep</i>), dan BLE.	Siti Amalia Nurfaidah
Role 3	Bertanggung jawab atas implementasi FreeRTOS pada <i>Gateway</i> , manajemen <i>Task</i> , logika BLE <i>Client</i> , dan kontrol aktuator.	Raka Arrayan Muttaqien
Role 4	Bertanggung jawab atas pengembangan <i>Backend Flask</i> , API deployment, visualisasi <i>Dashboard</i> Blynk, dan protokol komunikasi jaringan.	Naufal Hadi Rasikhin

Table 1. Roles and Responsibilities

1.5 TIMELINE AND MILESTONES

Berikut adalah jadwal pelaksanaan proyek yang dibagi menjadi empat tahapan utama:

Tahap	Deskripsi	Periode Waktu
Hardware Design Completion	Menyelesaikan desain rangkaian dan <i>schematic</i> Sensor Node & <i>Gateway</i> .	Minggu 1
Software Development	Pengembangan perangkat lunak menyeluruh meliputi <i>firmware</i> Sensor Node, manajemen tugas Gateway (FreeRTOS), dan server <i>Backend</i> (Flask API).	Minggu 1 sampai Minggu 2 (awal)
Integration and Testing	Menggabungkan <i>hardware</i> dan <i>software</i> , serta menguji komunikasi BLE, <i>Relay</i> , dan <i>cloud</i> .	Minggu 2 (pertengahan)
Final Product Assembly and Testing	Perakitan akhir dan pengujian lengkap untuk memastikan semua kriteria terpenuhi.	Minggu 2 (akhir)

Table 2. *Timeline and Milestones*

CHAPTER 2

IMPLEMENTATION

2.1 HARDWARE DESIGN AND SCHEMATIC

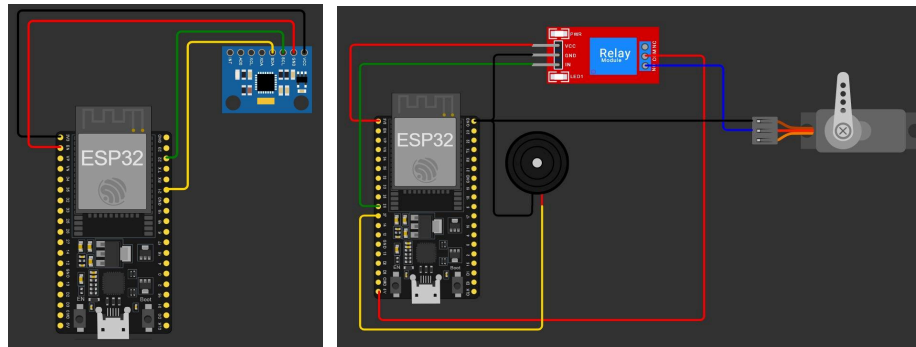


Figure 1 *SmartGuard's Hardware Design and Schematic*

Desain *hardware* sistem SmartGuard dibagi menjadi dua bagian utama, yaitu *Sensor Node* dan *Gateway Node*. Pada bagian *Sensor Node*, komponen utama yang digunakan adalah ESP32 Devkit dan sensor MPU6050. Sensor MPU6050 berfungsi untuk membaca getaran mesin melalui akselerometer 3 sumbu. Sensor ini terhubung ke ESP32 melalui komunikasi I2C, di mana pin SDA tersambung ke GPIO21 dan SCL ke GPIO22. ESP32 kemudian mengolah nilai akselerasi untuk menghitung indeks getaran, dan hasilnya dikirim secara nirkabel (*wireless*) menggunakan *Bluetooth Low Energy* (BLE). *Sensor Node* juga dilengkapi fitur *Deep Sleep*, sehingga perangkat otomatis masuk ke mode hemat daya ketika tidak mendeteksi getaran selama 30 detik.

Bagian *Gateway Node* menggunakan ESP32 Devkit sebagai pusat pengendalian. ESP32 pada *gateway* berperan sebagai *BLE Client* yang menerima data getaran dari *Sensor Node*, lalu menjalankan logika keamanan menggunakan dua *task* berbasis FreeRTOS. Pada *hardware gateway*, pin GPIO26 digunakan sebagai *output* untuk mengontrol Relay yang berfungsi sebagai saklar pemutus arus ke mesin. Sementara itu, GPIO27 terhubung ke Buzzer sebagai alarm indikator bahaya. Ketika nilai getaran melewati ambang batas, ESP32 akan memicu Relay untuk memutus jalur daya ke mesin (Motor DC) dan mengaktifkan Buzzer secara bersamaan. Selain itu, ESP32 *gateway* juga terhubung ke WiFi untuk mengirimkan data log ke server Flask serta menampilkan notifikasi status melalui aplikasi Blynk.

2.2 SOFTWARE DEVELOPMENT

a. Pemrograman *Sensor Node*

Program pada *Sensor Node* dikembangkan menggunakan pendekatan *Real-Time Operating System* (FreeRTOS) untuk memastikan manajemen tugas yang efisien. Pembacaan data sensor MPU6050 dilakukan secara manual melalui akses register I2C tanpa *library* pihak ketiga untuk mendapatkan data mentah akselerasi dan suhu secara *real-time*. Data akselerasi mentah dari register 0x3B dikonversi menjadi satuan m/s^2 , lalu dihitung menggunakan rumus magnitudo vektor untuk mendapatkan indeks getaran murni yang akurat. Data getaran dan suhu tersebut kemudian dikirimkan ke *Gateway* melalui notifikasi BLE dengan format string terformat.

Selain itu, algoritma manajemen daya pada Modul 8 diterapkan untuk efisiensi baterai. Sistem memantau aktivitas mesin, jika intensitas getaran berada di bawah ambang batas yaitu 1.2 m/s^2 selama 30 detik, *task sensor* akan mematikan modul komunikasi dan memicu ESP32 untuk masuk ke mode *Deep Sleep*. Perangkat akan bangun kembali atau *wake up* secara otomatis setiap 10 detik menggunakan *timer* internal untuk melakukan *sampling* ulang kondisi mesin.

b. Pemrograman *Gateway* (FreeRTOS Multitasking)

Gateway Node diimplementasikan sebagai *Central Controller* menggunakan FreeRTOS untuk manajemen tugas konkuren. Sistem ini membagi beban kerja ke dalam dua inti prosesor (*Dual Core*) untuk menjamin stabilitas:

- **Core 0 (TaskBLEManager)**

Core 0 bertanggung jawab penuh atas pemindaian dan pemeliharaan koneksi BLE. Data telemetri yang diterima dari *Sensor Node* berupa *string* terformat ("Getaran,Suhu") akan diparsing terlebih dahulu menjadi struktur data **SensorData** sebelum dimasukkan ke dalam antrian **sensorQueue**. Penggunaan struktur data ini memungkinkan pengiriman paket data yang kompleks antar *task* dengan aman.

- **Core 1 (TaskAppLogic)**

Core 1 bertindak sebagai otak keselamatan sistem. *Task* ini menunggu data dari antrian secara efisien. Saat data diterima, logika keselamatan lokal akan langsung membandingkan nilai getaran dengan ambang batas yaitu 5.5 m/s^2 . Jika bahaya terdeteksi, sistem memicu *Emergency Stop* (Relay OFF & Buzzer ON) secara instan. Selain itu, *task* ini juga menangani komunikasi *Cloud* dimana HTTP POST ke Flask dan telemetry Blynk, yang dilindungi oleh mekanisme Mutex (*wifiMutex*) untuk mencegah konflik akses pada modul WiFi saat *multitasking*.

- c. **Cloud Backend (Flask API)**

Subsistem *Cloud Backend* dikembangkan menggunakan *micro-framework* Python Flask yang berperan sebagai pusat pencatatan aktivitas atau *logging server* untuk sistem SmartGuard. Mekanisme komunikasi data diinisialisasi oleh *Gateway Node* yang dikonfigurasi untuk mengirimkan paket data telemetry dalam format JSON mencakup parameter getaran, suhu operasional, dan status bahaya ke *endpoint* **/api/log** melalui metode HTTP POST secara otomatis setiap kali terdeteksi anomali pada mesin. Untuk memastikan aksesibilitas data yang cepat, seluruh informasi insiden yang masuk disimpan dalam struktur penyimpanan sementara (*in-memory list*) pada sisi server.

Data riwayat ini selanjutnya dapat diakses kembali oleh administrator sistem melalui *endpoint* GET **/api/status** untuk keperluan audit teknis maupun evaluasi kronologis kejadian kerusakan mesin. Selain itu, untuk mencegah data riwayat hilang akibat *server* di-*restart*, Flask API dilengkapi dengan koneksi pada layanan database SQL tanpa server bernama Neon. Dengan adanya koneksi tersebut, walaupun terjadi masalah pada server API, data dari kejadian-kejadian sebelumnya tidak akan menghilang.

Sebagai pelengkap fungsi pencatatan log tersebut, arsitektur *cloud* sistem ini juga didukung oleh integrasi langsung dengan platform IoT Blynk untuk memfasilitasi interaksi pengguna secara *real-time*. Berbeda dengan

server Flask yang berfungsi sebagai arsip data kejadian, komunikasi antara *Gateway* dan *server* Blynk difokuskan pada visualisasi parameter mesin melalui antarmuka *dashboard* grafis di perangkat seluler pengguna. Integrasi ini tidak hanya menampilkan indikator status terkini, tetapi juga mendukung pengiriman notifikasi instan kepada operator saat kondisi kritis terjadi, sehingga tindakan mitigasi dapat diambil sesegera mungkin.

2.3 HARDWARE AND SOFTWARE INTEGRATION

Tahap integrasi sistem dilakukan dengan menggabungkan tiga subsistem utama, yaitu *Sensor Node*, *Gateway Node*, dan *Cloud Backend*. Tahap integrasi fisik dilakukan dengan menempelkan *Sensor Node* secara rigid pada *casing* motor DC menggunakan perekat industrial guna meminimalkan *noise* dan menstabilkan pembacaan sensor MPU6050. Sementara itu, *Gateway Node* ditempatkan dalam radius efektif BLE untuk menjamin kualitas sinyal, dengan *Relay* yang dihubungkan secara seri ke sumber daya motor sebagai mekanisme pemutusan arus elektronik.

Pada sisi *software*, integrasi komunikasi dijumpai oleh mekanisme *Service Discovery* protokol *Bluetooth Low Energy*. Saat sistem dinyalakan, *Gateway* yang bertindak sebagai *Client* secara otomatis memindai lingkungan sekitar untuk mencari *Sensor Node* yang memancarkan *Service UUID* spesifik yang telah dikonfigurasi pada *firmware*. Setelah *handshake* koneksi berhasil, *Gateway* segera melakukan *subscription* pada karakteristik notifikasi sensor. Hal ini memungkinkan data telemetri yang berisi nilai getaran dan suhu dikirimkan secara *real-time* ke *Gateway* setiap kali *Sensor Node* bangun dari mode tidur atau mendeteksi perubahan kondisi.

Tahap terakhir adalah validasi fungsional *end-to-end* untuk memastikan seluruh komponen bekerja sinergis. Pengujian dilakukan dengan mensimulasikan kondisi getaran berlebih pada motor. Sistem dinyatakan terintegrasi dengan baik apabila alur data berikut berjalan tanpa hambatan, yaitu akuisisi data oleh sensor, pengiriman data nirkabel ke *Gateway*, eksekusi logika keselamatan yaitu pemutusan *Relay* dan aktivasi *Buzzer* oleh *Gateway* dalam waktu kurang dari 1 detik, serta sinkronisasi log insiden ke *server* Flask dan pembaruan indikator visual pada *dashboard* aplikasi Blynk. Integrasi menyeluruh ini menjamin bahwa SmartGuard berfungsi sebagai satu kesatuan sistem proteksi yang responsif dan andal.

CHAPTER 3

TESTING AND EVALUATION

3.1 TESTING

Pengujian dilakukan secara komprehensif untuk memverifikasi fungsionalitas seluruh subsistem IOT 22-SmartGuard, mulai dari akurasi akuisisi data sensor, stabilitas komunikasi BLE, responsivitas aktuator keselamatan, hingga integrasi *cloud*. Skenario pengujian dibagi menjadi tiga kategori utama:

a. Sensor Node Testing

- Memverifikasi apakah akses register manual ke MPU6050 mampu membaca *raw data* akselerasi 3 sumbu dan suhu mesin secara valid.
- Menguji akurasi perhitungan magnitudo vektor getaran untuk memastikan nilai indeks yang dihasilkan sesuai dengan kondisi gangguan fisik yang diberikan.
- Memvalidasi mekanisme manajemen daya, yaitu memastikan *Sensor Node* masuk ke mode *Deep Sleep* setelah 30 detik *idle* dan bangun kembali atau *wake up* sesuai interval *timer* 10 detik.
- Menguji stabilitas pengiriman paket data yaitu Getaran dan Suhu melalui notifikasi BLE agar diterima secara periodik tanpa *packet loss*.

b. Gateway Node Testing

- Menguji kinerja *TaskBLEManager* di *Core 0* dalam memindai, menyaring UUID target, dan menjaga koneksi dengan *Sensor Node* secara otomatis.
- Memverifikasi *TaskAppLogic* di *Core 1* dalam memparsing data *string* menjadi *struct* dan memproses antrian (*Queue*) tanpa terjadi *blocking*.
- Menguji responsivitas *Relay* dan *Buzzer* dalam memutus daya mesin seketika saat nilai getaran melewati ambang batas bahaya.
- Melakukan *Stress Test* dengan mematikan router WiFi untuk memastikan logika keselamatan lokal tetap berfungsi normal meskipun koneksi internet terputus.

c. Cloud and Notification Testing

- Memverifikasi format pengiriman data JSON berisi getaran, suhu, dan status dari *Gateway* ke API Flask melalui endpoint */log*.
- Mengecek integritas data yang tersimpan pada *server* untuk memastikan riwayat kejadian dapat diakses kembali melalui *endpoint /status*.
- Menguji visualisasi *real-time* pada *Dashboard* Blynk serta memastikan notifikasi bahaya muncul tepat waktu.
- Menguji ketahanan mekanisme *Mutex* saat pengiriman data dilakukan dalam kondisi jaringan yang tidak stabil.

3.2 RESULT

Pengujian ini mencakup validasi menyeluruh terhadap integritas sinyal perangkat keras, responsivitas algoritma perangkat lunak, serta keandalan komunikasi nirkabel pada arsitektur *hybrid network*. Secara umum, sistem SmartGuard menunjukkan performa yang stabil dalam mendeteksi anomali getaran dan mengeksekusi protokol keselamatan otomatis. Berdasarkan skenario pengujian yang telah dilakukan, diperoleh hasil evaluasi kinerja sistem sebagai berikut:

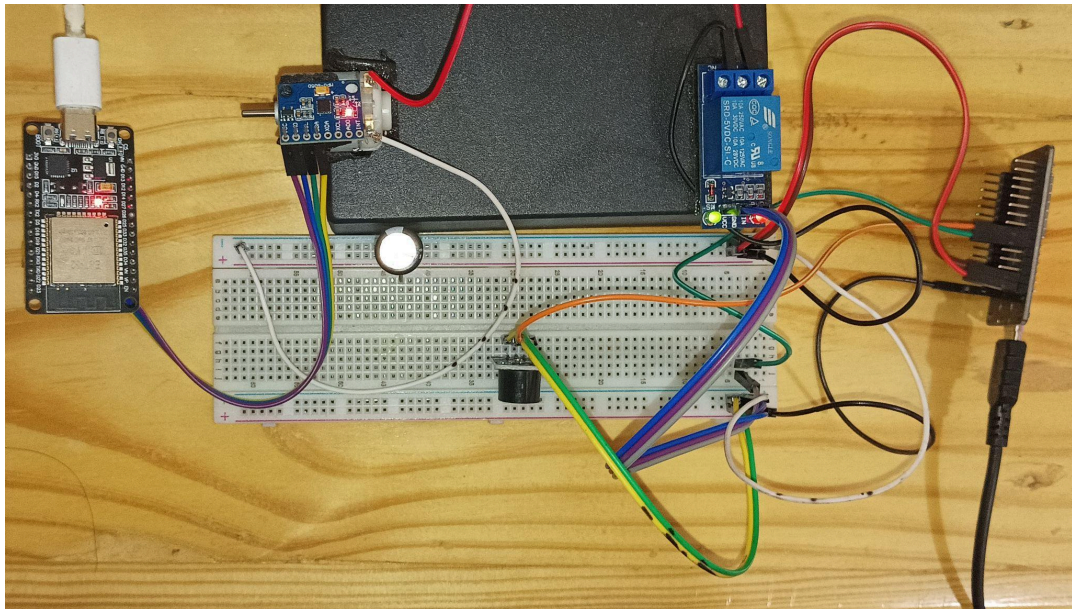


Figure 2 Testing Result

a. Sensor Node Result

- Akses *register* manual pada MPU6050 berhasil membaca data akselerasi dengan stabil. Deviasi pada sumbu Z akibat efek gravitasi berhasil dikompensasi menggunakan rumus magnitudo vektor, menghasilkan indeks getaran 0.00 - 0.05 m/s² saat kondisi diam.
- Mekanisme manajemen daya berfungsi sesuai konfigurasi. *Sensor Node* sukses memasuki mode *Deep Sleep* secara otomatis setelah 30 detik tidak ada aktivitas, dan bangun kembali setiap 10 detik berdasarkan *timer* internal.
- Transmisi data via BLE berjalan lancar dengan interval 1 detik. Paket data yang berisi Getaran dan Suhu berhasil dikirimkan tanpa *packet loss* yang signifikan.

b. Gateway Node Result

- *TaskBLEManager* pada Core 0 mampu memindai dan menjaga koneksi dengan sensor secara konsisten. Proses *parsing* data *string* menjadi *struct* berjalan tanpa membebani memori.
- *TaskAppLogic* pada Core 1 merespons data antrian (*Queue*) dengan cepat. Logika keselamatan memicu *Relay* untuk memutus daya mesin dalam waktu kurang dari 1 detik setelah ambang batas terlampaui.
- *Buzzer* berhasil aktif sebagai indikator audio saat status bahaya terdeteksi, dan mati otomatis saat sistem di *reset*.
- Fitur *Local Intelligence* terbukti handal, dimana sistem keselamatan (*Relay & Buzzer*) tetap bekerja normal memutus arus mesin meskipun koneksi WiFi atau *Router* dimatikan secara paksa.

c. Cloud and Notification Result

- *Gateway* berhasil mengirimkan paket data JSON ke Flask API secara *real-time* saat terjadi anomali.
- *Endpoint /status* menampilkan riwayat log insiden secara lengkap dan kronologis.

- *Dashboard* Blynk berhasil memvisualisasikan data Getaran dan Suhu secara akurat. Notifikasi bahaya atau *Push Notification* muncul tepat waktu di perangkat pengguna.
- Mekanisme *Mutex* pada *Gateway* berhasil mencegah konflik data, sehingga tidak ditemukan *error* pada pengiriman API meskipun dilakukan berulang kali.

3.3 EVALUATION

Berdasarkan analisis hasil pengujian, sistem **SmartGuard** dinilai telah memenuhi tujuan perancangan sebagai alat pemantauan getaran mesin yang responsif. Adapun evaluasi sistem adalah sebagai berikut:

a. Kelebihan

- Penggunaan rumus magnitudo pada *Sensor Node* terbukti efektif mengeliminasi kesalahan pembacaan akibat kemiringan orientasi sensor.
- Algoritma *Deep Sleep* berfungsi optimal, memungkinkan *Sensor Node* beroperasi lebih lama dengan baterai yang hanya aktif saat diperlukan.
- Implementasi arsitektur *multitasking* FreeRTOS pada *Gateway* menjamin proses pembacaan sensor dan eksekusi aktuator berjalan lancar tanpa *lag*.
- Protokol keamanan tetap berfungsi penuh memutus daya mesin saat bahaya, meskipun jaringan WiFi atau server *Cloud* mengalami gangguan.
- Integrasi dengan *Blynk* memberikan visibilitas status mesin dan suhu secara *real-time* kepada operator di mana saja.
- Integrasi API dengan database SQL untuk mencegah hilangnya data saat *server* mengalami gangguan.

b. Kekurangan

- Koneksi BLE memiliki keterbatasan jarak fisik, sehingga penempatan *Gateway* menjadi kurang fleksibel pada area yang luas.
- Modul *Relay* yang digunakan hanya mendukung arus rendah, sehingga diperlukan komponen tambahan untuk mesin industri berdaya besar.
- *Buzzer* hanya memiliki satu tingkat volume, yang mungkin kurang efektif terdengar di lingkungan industri dengan tingkat kebisingan tinggi atau *high decibel environment*.

CHAPTER 4

CONCLUSION

Proyek **SmartGuard** berhasil dikembangkan sebagai sistem pemantauan getaran mesin yang mampu mendeteksi kondisi berbahaya secara cepat dan akurat. Dengan memanfaatkan sensor MPU6050, komunikasi BLE, dan pengaturan tugas berbasis FreeRTOS, sistem dapat merespons kondisi bahaya secara otomatis dengan memutus daya mesin dan mengaktifkan alarm. Semua proses ini berjalan secara *real-time* dan tetap stabil meskipun koneksi WiFi mengalami gangguan.

Hasil pengujian menunjukkan bahwa **SmartGuard** dapat bekerja sesuai dengan tujuan yang direncanakan. Sistem mampu membaca getaran mesin dengan konsisten, menjaga koneksi BLE dengan baik, serta mengirimkan notifikasi dan *log* kejadian ke *server* Flask dan aplikasi Blynk tanpa hambatan. Selain itu, fitur *Deep Sleep* pada *Sensor Node* membantu menghemat penggunaan daya sehingga perangkat lebih efisien untuk digunakan dalam waktu lama.

Secara keseluruhan, **SmartGuard** telah memenuhi seluruh *acceptance criteria* yang ditetapkan dan memberikan dasar yang kuat untuk pengembangan berikutnya. Walaupun masih ada beberapa kekurangan, seperti jangkauan BLE yang terbatas dan kapasitas *relay* yang perlu ditingkatkan, sistem ini tetap menunjukkan performa yang baik. Dengan penyempurnaan pada sisi komunikasi dan penyimpanan data, SmartGuard dapat menjadi sistem pemantauan mesin yang lebih lengkap, stabil, dan siap digunakan di berbagai kebutuhan industri.

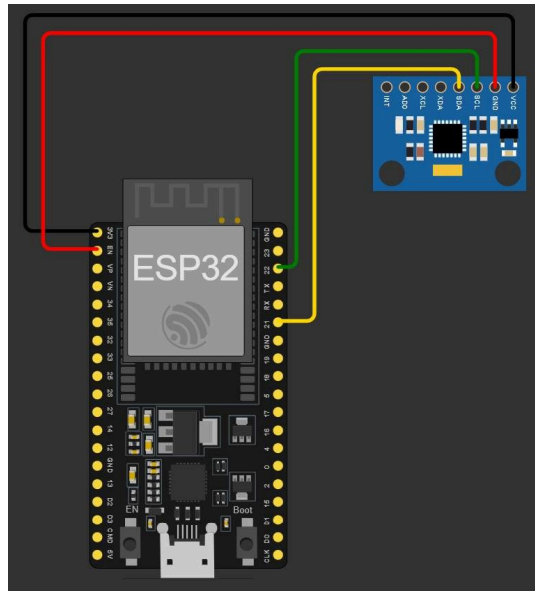
REFERENCES

- Random Nerd Tutorials, “ESP32 MPU-6050 Accelerometer and Gyroscope (Arduino) | Random Nerd Tutorials,” *Random Nerd Tutorials*, Jan. 12, 2021. [Online]. Available: [ESP32 MPU-6050 Accelerometer and Gyroscope \(Arduino\) | Random Nerd Tutorials](#) [Accessed: Dec. 07, 2025]
- Random Nerd Tutorials, “ESP32 I2C Communication: Set Pins, Multiple Bus Interfaces and Peripherals | Random Nerd Tutorials,” *Random Nerd Tutorials*, Oct. 02, 2019. [Online]. Available: [ESP32 I2C Communication: Set Pins, Multiple Bus Interfaces and Peripherals | Random Nerd Tutorials](#). [Accessed: Dec. 07, 2025].
- Blynk, “Introduction - Blynk Documentation,” Blynk.io, 2022. [Online]. Available: [Introduction | Blynk Documentation](#). [Accessed: Dec. 08, 2025].
- Espressif Systems, “ESP-BLE-MESH - ESP32 - — ESP-IDF Programming Guide v5.2.3 documentation,” Espressif.com, 2016. [Online]. Available: [ESP-BLE-MESH - ESP32 - — ESP-IDF Programming Guide v5.5.1 documentation](#). [Accessed: Dec. 08, 2025].
- Random Nerd Tutorials, “Arduino Guide for MPU-6050 Accelerometer and Gyroscope | Random Nerd Tutorials,” Random Nerd Tutorials, Feb. 16, 2021. [Online]. Available: [Arduino Guide for MPU-6050 Accelerometer and Gyroscope | Random Nerd Tutorials](#). [Accessed: Dec. 08, 2025].

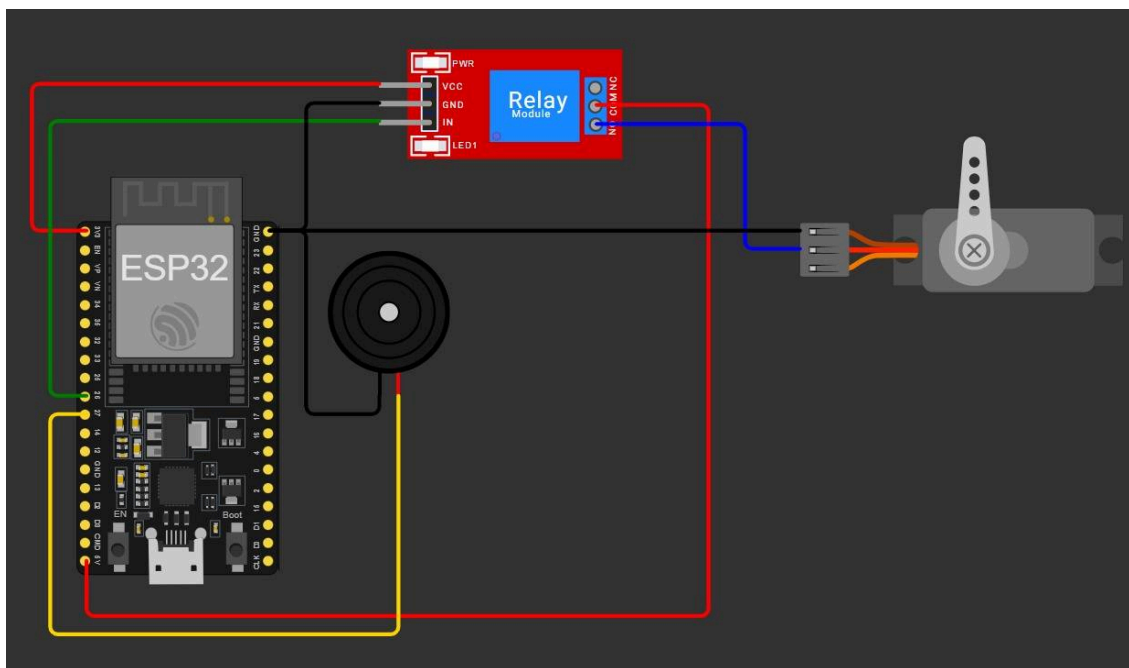
APPENDICES

Appendix A: Project Schematic

Sensor Node

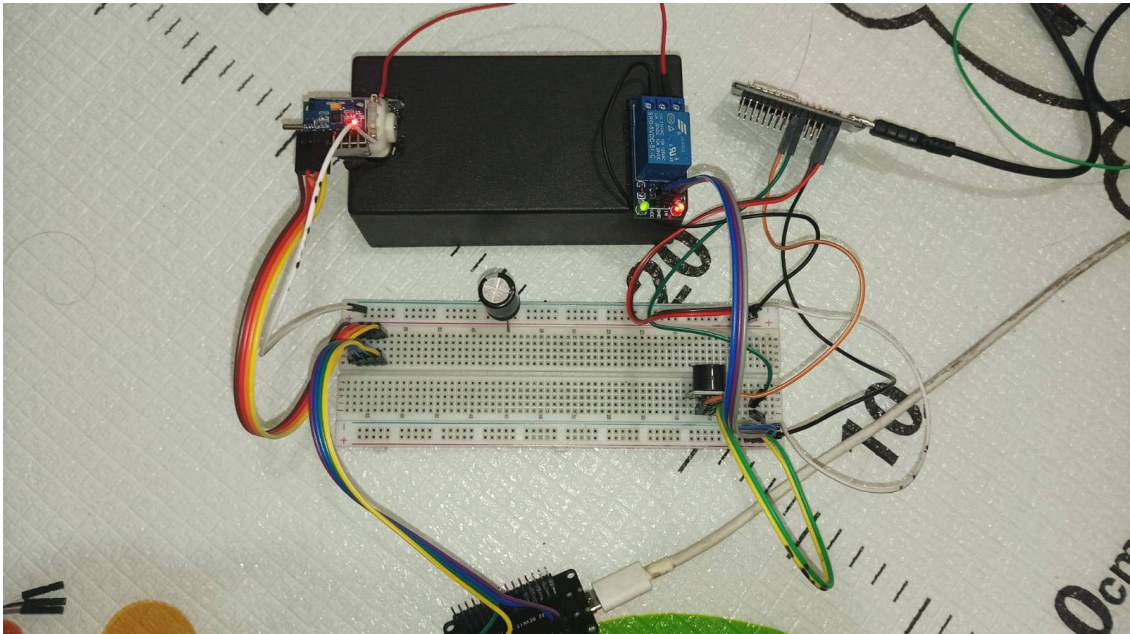


Gateway Node



Appendix B: Documentation

Photos



Appendix C: Demonstration

Link

<https://youtu.be/MpqkfAPGV-E>